

Stroke Studio — Online Art Gallery & Marketplace

February 2026 · Full-Stack Web App · Node.js · Express · MongoDB · Cloudinary

Anshumaan Sai Patnaik

Overview

Stroke Studio is a full-stack web application that serves as an online art gallery and marketplace. Visitors can browse a curated collection of paintings; registered users get a personal dashboard where they can manage a cart, a wishlist, and a record of purchased artworks, edit their profile, and upload a custom avatar.

Unlike the earlier, single-file builds, Stroke Studio is organised as a layered **MVC backend** with a clear separation between configuration, data models, route handlers, and middleware. It is backed by MongoDB through Mongoose, uses session-based authentication, integrates Cloudinary for image hosting, and seeds its catalogue from a real public museum API — making it the most complete project in this collection.

Technology Stack

Node.js + Express	Application server and routing, organised as a modular MVC backend.
MongoDB + Mongoose	Document database with schema-based models for users and paintings.
express-session	Session-based authentication and per-user access control.
Cloudinary + Multer	Avatar uploads — Multer receives the file, Cloudinary hosts it and returns a URL.
Axios	Fetches source artwork data from an external museum API during seeding.
EJS	Server-side templates for the home, gallery, and user dashboard pages.
Faker · UUID · Dotenv	Sample-data generation, unique IDs, and environment-based secrets.

Architecture

The backend follows a conventional MVC layout, with each responsibility in its own folder. This keeps the route files thin, pushes logic into controllers, and isolates external services behind small config modules:

- **config/** — the MongoDB connection and the Cloudinary client setup.
- **models/** — Mongoose schemas for *User* and *Painting*.
- **controllers/** — the actual logic for home, gallery, and user actions.
- **routes/** — modular routers mounted under `/home`, `/gallery`, and `/user`.
- **middleware/** — the Multer upload handler.
- **init/** — one-off scripts that seed the database with paintings and users.

Express is configured with JSON and URL-encoded body parsing, static asset serving for the front-end and uploads, EJS as the view engine, and an **express-session** layer that the controllers use to track the logged-in user.

Routes

METHOD	PATH	PURPOSE
GET	/home	Landing page — recent and randomly sampled works.
GET	/gallery	Public gallery of paintings.
POST	/user/register	Create an account and start a session.
POST	/user/signin	Authenticate against stored credentials.
GET	/user/signout	Destroy the session.
GET	/user/:id	Personal dashboard (access-guarded).
PATCH	/user/:id/update-profile	Verify password and update profile fields.
PATCH	/user/:id/update-list	Add/remove cart and wishlist items.
POST	/user/upload-avatar	Upload an avatar via Cloudinary.
DELETE	/user/:id/delete	Delete the account.

Data Models

Two Mongoose schemas hold the application's data, with the user's collections modelled as references to painting documents:

- **Painting** — the artwork catalogue: image path, hosted image URL, a unique image ID, plus *title*, *artist*, *year*, *style*, *medium*, *dimensions*, and *price*.
- **User** — account fields (ID, username, email, password, avatar) along with three arrays — *cartItems*, *wishList*, and *purchased* — each holding **ObjectId references** to Painting documents rather than copies. This keeps a single source of truth for each artwork and lets a user's collections be populated on demand.

Key Functionality

Authentication & access control

Registration creates a user, assigns a UUID, and stores the user's ID in the session; sign-in looks the user up by credentials and does the same; sign-out destroys the session. Pages are guarded by that session state — the dashboard route only renders when the session's user ID matches the requested `:id`, and the public home and gallery pages redirect already-logged-in users straight to their own dashboard.

Cart & wishlist

A single endpoint handles all four cart and wishlist mutations, dispatched by an **action** field. Each maps to an atomic MongoDB array update — `$addToSet` to add an item without creating duplicates, and `$pull` to remove it — so the operations are safe and idempotent without reading the document first.

```
// Cart & wishlist updates – atomic array operations keyed by action
const id = req.session.user.userID;
const { action, paintingId } = req.body;

if (action === 'cart-add')
  await User.updateOne({ userID: id }, { $addToSet: { cartItems: paintingId } });
if (action === 'cart-remove')
  await User.updateOne({ userID: id }, { $pull: { cartItems: paintingId } });
if (action === 'wish-add')
  await User.updateOne({ userID: id }, { $addToSet: { wishList: paintingId } });
if (action === 'wish-remove')
  await User.updateOne({ userID: id }, { $pull: { wishList: paintingId } });
```

Profile & avatar uploads

Profile updates re-verify the user's current password before applying changes and only write the fields that were actually provided (username always, password and avatar optionally). Avatar uploads go through Multer, which receives the file and hands its path to Cloudinary; the returned secure URL is what gets stored on the user, so the app never serves large image files itself.

Seeding the catalogue from a public API

Rather than inventing fake artwork, the catalogue is seeded from the Art Institute of Chicago's public API. The seeder requests public-domain paintings created from 1950 onward, filters out entries without an image, missing dimensions, or by a short list of famous artists, normalises each record into the Painting schema (including parsing the dimension string and assigning a randomised price), and saves a fixed number of works. Users are then seeded separately, each given a random selection of cart, wishlist, and purchased paintings.

```
// Seed the catalogue from the Art Institute of Chicago public API
const response = await axios.get(API_URL, {
  params: { q: "painting", fields: FIELDS, limit: 100,
    query: JSON.stringify({
      term: { is_public_domain: true },
      range: { date_start: { gte: 1950 } }
    }) }
});

for (const artwork of response.data.data) {
  if (!artwork.image_id) continue; // no image
  if (famousArtists.includes(artwork.artist_title)) continue;
  await new Painting(transformArtwork(artwork, rawDims)).save();
  if (++savedCount === TOTAL_PAINTINGS) break;
}
```